SHASU VATHANAN

SHASU VATHANAN - GEN AI - PRODUCT MANAGER

GEN AI ARCHITECT JOURNEY · PHASE-001

Student Grade System

Python fundamentals, validated end to end

Phase	Phase-001 - Python Fundamentals
Folder	Phase-001-Programming-Foundations
What it is	A terminal program that turns a mark into a letter grade
Level	Beginner
Tools	Python 3 · Windows PowerShell
Single file	grade_system.py

Shasu Vathanan - GEN AI - Product Manager

SHASUVATHANAN.COM · linkedin.com/in/shasuvathanan

1. What this phase builds

A terminal-based Python program that converts a mark from 0 to 100 into a letter grade.

It is deliberately small. The value is not the arithmetic - it is that every path through the program is accounted for. A mark that is text, a mark that is negative, a mark above 100, and a mark that is not a finite number are all handled before any grading happens.

> **Why start here**
>
> Input validation, clear failure messages, and a single responsibility per function are the same habits that later keep a Streamlit app from crashing and an API from returning a wrong answer with a 200 status. This phase establishes them while the program is still small enough to hold in your head.

2. Run it

POWERSHELL

```
python grade_system.py
```

The program prompts once, prints the result, and exits. There is nothing to install - it uses only the Python standard library.

3. The grade scale

Mark	Grade
90 - 100	A
80 - 89.999...	B
70 - 79.999...	C
60 - 69.999...	D
Below 60	E

Boundaries are inclusive at the lower end. A mark of exactly 90 is an A, exactly 80 is a B, and so on. Decimal marks are supported throughout, which is why the upper bounds are written as **89.999...** rather than **89**.

4. Example runs

TEXT

```
Enter your mark (0-100): 95
Mark: 95 -> Grade: A

Enter your mark (0-100): 85
Mark: 85 -> Grade: B

Enter your mark (0-100): 59
Mark: 59 -> Grade: E
```

5. How invalid input is handled

Invalid input is handled without crashing. Non-numeric values produce a clear request to enter a number, while values outside 0-100 - including non-finite values such as **nan** - produce a clear range error.

Input	What happens	Why
85	Mark: 85 -> Grade: B	A valid mark inside the range
85.5	Mark: 85.5 -> Grade: B	Decimals are accepted
abc	Invalid input: please enter a number between 0 and 100.	**float()** raises **ValueError**, which is caught
-5	Invalid mark: please enter a number between 0 and 100.	Outside the accepted range
120	Invalid mark: please enter a number between 0 and 100.	Outside the accepted range
nan	Invalid mark: please enter a number between 0 and 100.	**math.isfinite()** rejects it before any comparison

The nan case is the interesting one

float("nan") succeeds, so the **try/except** alone would let it through. Worse, every comparison against nan returns **False**, so **0 <= mark <= 100** is False and the range check happens to catch it - but only by accident. The explicit **math.isfinite()** test makes the intent deliberate rather than lucky.

6. The complete source

PYTHON

```python
"""Convert a student mark from 0 to 100 into a letter grade."""

import math


def calculate_grade(mark: float) -> str:
    """Return the letter grade for a validated mark."""
    if mark >= 90:
        return "A"
    if mark >= 80:
        return "B"
    if mark >= 70:
        return "C"
    if mark >= 60:
        return "D"
    return "E"


def main() -> None:
    """Read a mark, validate it, and display the matching grade."""
    entered_mark = input("Enter your mark (0-100): ").strip()

    try:
        mark = float(entered_mark)
    except ValueError:
        print("Invalid input: please enter a number between 0 and 100.")
        return

    if not math.isfinite(mark) or not 0 <= mark <= 100:
        print("Invalid mark: please enter a number between 0 and 100.")
        return

    display_mark = f"{mark:g}"
    grade = calculate_grade(mark)
    print(f"Mark: {display_mark} -> Grade: {grade}")


if __name__ == "__main__":
    main()
```

7. Design notes

Two functions, two jobs

calculate_grade() only grades. It assumes its input is already valid, which is why it has no error handling and no printing. **main()** owns everything to do with the outside world: reading input, validating it, and displaying the result. That separation is what makes the grading logic trivially reusable - and it is reused unchanged in the Phase-004-FastAPI-And-Streamlit Streamlit app.

Why the ladder has no upper bounds

Each **if** returns immediately, so by the time execution reaches **mark >= 80** the mark is already known to be under 90. Writing **80 <= mark < 90** would be redundant and would leave a gap the moment a boundary changed.

The :g format

f"{mark:g}" prints **85** rather than **85.0** for whole numbers, while still showing decimals when they exist. Small detail, but it is the difference between output that looks finished and output that looks like a debug print.

Next

Phase-002-GenAI-Python-Toolkit builds out the Python concepts behind this program - types, collections, control flow, functions, errors, and files.

Shasu Vathanan - GEN AI - Product Manager
SHASUVATHANAN.COM · linkedin.com/in/shasuvathanan